

UNIT 2

COMPUTATIONAL THINKING & ALGORITHMS

9TH COMPUTER SCIENCE FEDERAL BOARD

AWINASH GOSWAMI

AGENDA

INTRODUCTION

INTRODUCTION

- A **computer** is a machine that solves problems by **accepting inputs, performing operations, and presenting outputs**.
- Computers **can't think** — they only perform operations written by us as **algorithms** and fed in as **programs**.
- You can't program a computer to solve something you don't know how to solve yourself. So, you must first **understand the problem, create a solution, and instruct the computer** accordingly.
- Computers vary in size and capacity, but computation is mainly based on the **processor and main memory**.

PROBLEMS

A problem is a challenge or situation that needs to be overcome using some action. The problems are present in all fields such as economic, healthcare, education, transportation, internet, biology and many more.

Economic

What to produce?

How to manage limited budgets?

Healthcare

How to expand access to medicines?

How to stop infectious diseases?

Education

How to increase literacy rate in Pakistan?

What subject areas of education should be focused on more?

Transportation

Why is the number of vehicles growing up?

Why the accidents on motorways are increasing?

Internet

How to secure mobile banking transactions?

Why do websites show unwanted advertisements?

Biology

How do 5G cell phone signals harm humans?

How do trees give earth all its oxygen?

IDENTIFYING A COMPUTING PROBLEM

In computer science, a **problem** is a challenge or situation that needs to be overcome using some action. A **computing problem** is solved **step-by-step using computation** (arithmetic or logical). These problems usually have a **well-defined input** and **desired output properties**. Below are the types of computing problems:

- *Decision Problems*
- *Search Problems*
- *Counting Problems*

Decision Problems – A problem where the input requires a **binary response: Yes or No** (or True/False). Complex cases may involve **multiple decision factors**.

Examples:

- *Is a given number **odd or even**?*
- *Is a given number a **prime number**?*
- *Does the sequence contain "**aa**"?*

IDENTIFYING A COMPUTING PROBLEM

Search Problems – In science and engineering, many problems are solved using **search**. We have a **set of objects** and search for the solution among them. These are often represented using **graphs** (nodes + links).

Three things needed to solve:

- **Initial State** – *The node where the search starts.*
- **Operations** – *The moves that transition from one node to another.*
- **Goal** – *The target or end condition.*

Examples:

- **Route Finding Problem** – *Find a route from city **S** to city **T** on a map (nodes = cities, links = routes).*
- **Eight Puzzle Problem** – *A **3×3 board** with 8 numbered tiles and 1 empty space. Slide tiles (left, right, up, down) to match the **final configuration**.*

IDENTIFYING A COMPUTING PROBLEM

Counting Problems work on the principle that if one event has **A choices** and another has **B choices**, the total unique combinations = **$A \times B$** .

- **Example 1:**

- *Shirts = 5, Pants = 3*
- *Total outfits = $5 \times 3 = 15$*

- **Example 2:**

- *Buying a computer system:*
- *Monitors = 4*
- *Keyboards = 2*
- *Computers = 4*
- *Printers = 3*

Total possible systems = $4 \times 2 \times 4 \times 3 = 96$

PROBLEM SOLVING

Problem Solving is the process of analyzing a situation and responding to generate a solution.

4 Steps for Simple Problems:

- *Define and analyze* – What is the problem and why is it happening?
- *Design a plan* – What will we do? (Algorithm)
- *Implement the plan* – Code it using a programming language.
- *Evaluate* – Did our plan work?

6 Steps for Complex Problems:

- *Define and analyze* the problem.
- *Decompose* – Break it into manageable sub-problems.
- *Identify potential plans* for each sub-problem (Plan A, B, C).
- *Select and design* the best plan.
- *Implement* that plan.
- *Evaluate*.

Before acting, a problem must be **thoroughly investigated**. Defining a problem requires **clarity** and **specified objectives**.

INPUT-PROCESSING-OUTPUT (I-P-O) MODEL

Computer systems work on the **Input-Process-Output (IPO) model**. Analyzing a problem means breaking it into three components:

- **Input** – *The requirements/data given to the system from the environment.*
- **Processing** – *The operations/computations the system performs based on the input.*
- **Output** – *The result presented back to the environment.*

Correct identification of all three is required to solve a problem properly.

COMPUTATIONAL THINKING

Computational Thinking is the ability to think in a certain way to solve problems using computation. **Tip:** Before learning programming, understand computational thinking.

Importance:

- *Helps reflect clearly on a problem and define a **repeatable solution**.*
- *Goal is not just to solve one instance, but to build an **automated system** that solves all similar problems.*
- *It is an **extension of logical thinking**.*

Properties:

- **Decomposition** – Breaking large problems into smaller, manageable **sub-problems** and solving them one by one.
- **Abstraction** – Removing **unnecessary details** to focus on essential information.
- **Pattern Recognition** – Looking for **patterns or similarities** with previously solved problems.
- **Algorithm Design** – Creating a **step-by-step plan** to solve the problem.

PRINCIPLES OF COMPUTATIONAL THINKING

Logical Thinking means analyzing a situation using **reason** to reach **sensible decisions**. It requires gathering **relevant facts** and choosing the best path based on **evidence**.

Example: You enter your home and see water on the floor. What happened? You use facts and reasoning to figure out the cause.

Key Point: Logical thinking is fundamental to **computational thinking**. Computers use logic in computation, but they **cannot think logically on their own** — they must be **programmed** to do so.

ALGORITHMIC THINKING

Algorithmic Thinking is about problem-solving. Computers do nothing by magic — algorithmic thinking is a way of **getting to a solution** by identifying the **steps needed** and implementing them in a **logical and efficient manner**. It involves **breaking complex problems into smaller parts** and solving them one at a time.

Example Algorithm – Find the Largest of Three Numbers:

Step 1: Let $A = 10$, $B = 20$, $C = 30$.

Step 2: Check if A is the largest ($A > B$ and $A > C$).

Step 3: If A is largest, stop. Otherwise, continue.

Step 4: Check if B is the largest ($B > A$ and $B > C$).

Step 5: If B is largest, stop. Otherwise, continue.

Step 6: C is the largest.

METHODS TO DESIGN A SOLUTION

Once the problem is fully understood, the next step is to **design a solution**. This means defining **how the software will meet the requirements and objectives** of the problem.

Two methods used to design a solution:

Flowcharts

Concept Maps

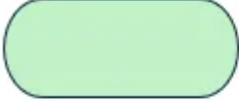
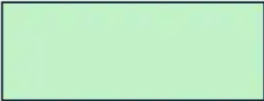
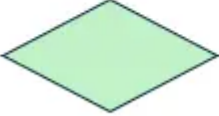
Flowcharts

A flowchart is a **diagrammatic representation of an algorithm**. It describes **what operations are required** to solve a given problem.

Importance of Flowcharts:

- Illustrates the **sequence of operations** as a diagram.
- Programmers draw flowcharts **before writing programs**.

METHODS TO DESIGN A SOLUTION: FLOWCHART

Symbol	Name	Function
	Oval	Represents the start or end of a process
	Rectangle	Denotes a process or operation step
	Arrow	Indicates the flow between steps
	Diamond	Signifies a point requiring a yes/no
	Parallelogram	Used for input or output operations